

Zugang zum LLM-Service der HAW Hamburg

Anleitung für Studierende: Wie ihr einen lokal gehosteten LLM-Service über eine OpenAI-kompatible API nutzt – kostenlos, datenschutzfreundlich, ohne Anmeldung bei externen Anbietern.

Was ihr bekommt

Auf dem Rechenzentrum der HAW läuft ein Open-Source-Sprachmodell (Qwen2.5-7B-Instruct) hinter einer OpenAI-kompatiblen Schnittstelle. Das heißt: Euer Code, der für OpenAI/Anthropic/Co. geschrieben ist, funktioniert hier ohne Anpassung – ihr müsst nur `base_url` und `api_key` umstellen.

Pro Studierenden:

- Persönlicher API-Key (`sk-...`)
- Token-Budget pro Monat
- Rate-Limits (Requests pro Minute, Tokens pro Minute)
- Logging eurer Nutzung im Backend

Was ihr braucht

1. **Euren persönlichen API-Key.** Den habt ihr per Hochschul-Mail von eurer Lehrperson bekommen. Format: `sk-` gefolgt von ca. 40 Hex-Zeichen. Behandelt ihn wie ein Passwort – nicht in Git committen, nicht in Slack posten.
2. **Netzwerkzugang zum Hochschulnetz.** Aus dem WLAN auf dem Campus geht es direkt; von zu Hause braucht ihr das Hochschul-VPN.
3. **Python ≥ 3.9** mit pip, oder ein anderes Tool eurer Wahl.

Ihr braucht **kein** kubectl und **keinen** Cluster-Zugang – der Service ist über eine normale HTTPS-URL erreichbar.

Schritt 1: Smoke-Test mit `curl`

Bevor ihr Code schreibt, prüft kurz, dass alles funktioniert:

```
# Euren Key in eine Variable setzen
export OPENAI_API_KEY="sk-EuerKeyHier"

# Testen, ob der Service antwortet
curl https://llm.inf.haw-hamburg.de/v1/models \
-H "Authorization: Bearer $OPENAI_API_KEY"
```

Erwartete Antwort: Ein JSON mit einem Eintrag `qwen2.5-7b`. Wenn ihr das seht, ist alles bereit.

Erster echter Inferenz-Call:

```
curl https://llm.inf.haw-hamburg.de/v1/chat/completions \
-H "Authorization: Bearer $OPENAI_API_KEY" \
-H "Content-Type: application/json" \
-d '{
  "model": "qwen2.5-7b",
  "messages": [{"role": "user", "content": "Erkläre Kubernetes in einem Satz."}],
  "max_tokens": 100
}'
```

Wenn ein deutscher Antwortsatz zurückkommt, läuft die ganze Kette.

Schritt 2: Python-Setup

Drei verbreitete Wege, je nachdem was ihr macht.

Variante A: OpenAI-Python-SDK (empfohlen für die meisten Fälle)

Das offizielle SDK funktioniert direkt – LiteLLM ist ja OpenAI-kompatibel.

```
pip install openai
```

```
from openai import OpenAI

client = OpenAI(
    base_url="https://llm.inf.haw-hamburg.de/v1",
    api_key="sk-EuerKeyHier", # besser: aus Umgebungsvariable, siehe unten
)

response = client.chat.completions.create(
    model="qwen2.5-7b",
    messages=[
        {"role": "system", "content": "Du bist ein hilfreicher Assistent."},
        {"role": "user", "content": "Was ist der Unterschied zwischen RAG und Fine-Tuning?"},
    ],
    max_tokens=300,
    temperature=0.7,
)

print(response.choices[0].message.content)
```

Besser ohne Hardcoding des Keys: Setzt zwei Umgebungsvariablen, dann erkennt das SDK sie automatisch:

```
export OPENAI_BASE_URL="https://llm.inf.haw-hamburg.de/v1"
export OPENAI_API_KEY="sk-EuerKeyHier"
```

```
from openai import OpenAI
client = OpenAI() # liest beide Variablen automatisch

response = client.chat.completions.create(
    model="qwen2.5-7b",
    messages=[{"role": "user", "content": "Hallo!"}],
)

print(response.choices[0].message.content)
```

Variante B: LangChain

```
pip install langchain langchain-openai
```

```
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(
    base_url="https://llm.inf.haw-hamburg.de/v1",
    api_key="sk-EuerKeyHier",
    model="qwen2.5-7b",
    temperature=0.7,
)

response = llm.invoke("Erkläre RAG in zwei Sätzen.")
print(response.content)
```

Variante C: Streaming (wenn ihr lange Antworten live anzeigen wollt)

```
from openai import OpenAI
client = OpenAI()

stream = client.chat.completions.create(
    model="qwen2.5-7b",
    messages=[{"role": "user", "content": "Schreibe ein kurzes Gedicht über Kubernetes."}],
    stream=True,
    max_tokens=300,
)

for chunk in stream:
    if chunk.choices[0].delta.content:
        print(chunk.choices[0].delta.content, end="", flush=True)
print()
```

Eure Limits im Auge behalten

Eure persönlichen Limits sind:

- **Token-Budget pro Monat** – ein Spend-Cap, das LiteLLM intern verfolgt. Wenn ihr über das Limit kommt, werden Requests abgelehnt, bis das nächste Abrechnungsfenster beginnt.
- **Rate-Limits**: maximal X Requests pro Minute (RPM) und Y Tokens pro Minute (TPM). Wenn ihr zu viele Requests in kurzer Zeit abschickt, bekommt ihr HTTP 429 zurück – einfach kurz warten und erneut versuchen.

Was tun, wenn ihr an die Grenze stoßt: Das ist nicht zwingend ein Fehler, sondern oft ein Hinweis auf einen Bug. Typische Ursachen:

- **Endlosschleife in einem Agenten**: Eure Tool-Use-Logik ruft das Modell ohne Abbruchbedingung auf
- **Vergessenes `max_tokens`**: Das Modell generiert tausende Token, obwohl ihr nur eine kurze Antwort braucht
- **Zu lange Konversationshistorie**: Bei langer Chat-History wird jedes Request immer teurer, weil der ganze Kontext mitgeschickt wird

Beim Bauen von Agenten ist es gute Praxis, einen Step-Counter und ein Token-Budget pro Session zu setzen – genau das werdet ihr in der Vorlesung zu "Agentic AI Engineering" diskutieren.

Häufige Probleme

Could not resolve host oder Connection-Timeout

Ihr seid wahrscheinlich nicht im Hochschulnetz. Vom Campus-WLAN aus geht es direkt; von zu Hause oder unterwegs braucht ihr das Hochschul-VPN.

401 Unauthorized mit "Malformed API Key"

Der `Authorization`-Header ist nicht korrekt aufgebaut. Häufigste Ursache: Die Umgebungsvariable `OPENAI_API_KEY` ist im aktuellen Terminal nicht gesetzt. Prüfen mit:

```
echo "Key: '$OPENAI_API_KEY'"
```

Wenn dort nur `Key: ''` erscheint, einfach erneut exportieren.

429 Too Many Requests

Rate-Limit erreicht. Kurz warten (60 Sekunden reichen meistens) und erneut versuchen. Wenn ihr regelmäßig in die Limits lauft, prüft euren Code auf die in "Eure Limits im Auge behalten" beschriebenen Probleme.

400 Bad Request mit "context length exceeded"

Eure Konversation ist länger als der maximale Kontext (8192 Token bei diesem Setup). Lösungen:

- Konversationshistorie auf die letzten N Nachrichten kürzen
- System-Prompt verkürzen
- Bei RAG: weniger oder kleinere Dokumente in den Kontext laden

TLS-/Zertifikatsfehler

Eigentlich sollte das Zertifikat von eurem Browser/Python ohne weiteres akzeptiert werden. Falls doch nicht: Meldet euch bei eurer Lehrperson – das deutet auf ein Konfigurationsproblem im Cluster hin, nicht auf einen Fehler in eurem Code.

Bitte nicht mit `verify=False` umgehen, das deaktiviert die Server-Authentifizierung und ist ein didaktisch schlechtes Vorbild.

Der Service ist offline

Wenn der Smoke-Test mit `curl` Connection-Refused oder eine 5xx-Antwort liefert, ist es ein Infrastruktur-Problem. Meldet euch bei eurer Lehrperson – das ist nichts, was ihr selbst beheben könnt oder solltet.

Kein Zugriff auf das Internet aus dem Modell

Damit ihr es wisst: Qwen2.5-7B hat kein Wissen über aktuelle Ereignisse und kann nicht im Web suchen. Wenn ihr aktuelle Infos braucht, müsst ihr selbst Tool-Use bauen (Web-Search-Funktion, RAG mit aktuellen Quellen, etc.). Das ist Teil dessen, was ihr in den Übungen lernen sollt.

Datenschutz und Privatsphäre

Im Gegensatz zu kommerziellen LLM-Anbietern verlässt **kein einziger Token** dieses Setup die HAW-Infrastruktur. Eure Anfragen gehen vom Laptop direkt in den Cluster und werden dort verarbeitet. Es gibt aber zwei Punkte, die ihr wissen solltet:

- LiteLLM loggt zu jedem Request: Zeitstempel, euren User-ID, Modell, Token-Anzahl, Response-Zeit. **Nicht** den Inhalt eurer Prompts.

- Wenn ihr in der Vorlesung explizit gebeten werdet, eure Logs/Traces hochzuladen (z.B. für Debugging einer Übungsabgabe), kann eure Lehrperson eure Inhalte sehen.

Trotzdem: Behandelt diesen Service nicht als sicheren Kanal für sensible Daten. Personenbezogene Daten Dritter, Klausurfragen, Geschäftsgeheimnisse aus Praktikumsbetrieben – nichts davon gehört in einen Prompt, auch nicht in einen lokalen.

Wenn etwas nicht klappt

Bevor ihr fragt, einmal kurz selbst checken:

1. Seid ihr im Hochschulnetz? (`ping llm.inf.haw-hamburg.de`)
2. Ist `OPENAI_API_KEY` im aktuellen Terminal gesetzt? (`echo "$OPENAI_API_KEY"`)
3. Funktioniert der einfache `curl https://llm.inf.haw-hamburg.de/v1/models` -Test mit dem Bearer-Header?

Wenn alle drei Punkte funktionieren und euer Code trotzdem fehlschlägt, ist es ein Code-Problem (Library, Modellname, Parameter). Wenn schon einer der drei Punkte fehlschlägt, ist es ein Infrastruktur- oder Setup-Problem – und dann ist eure Lehrperson die richtige Anlaufstelle.

Viel Spaß beim Bauen!